

## 1. MDP 基础概念

**MDP: Markov Decision Process** a model for sequential decision-making under uncertainty

**Markov property** the next state depends only on the current state and action, not on the full history

**transition function**  $T(s,a,s')=P(s' | s,a)$  gives the probability of reaching the next state after taking an action

**reward**  $R(s,a,s')$  the immediate feedback received after an action

**Policy** specifies which action the agent should take in each state

**Utility**  $U_{\pi}(s)=E[r_1+\gamma r_2+\gamma^2 r_3+\dots]$  the expected long-term value of a state 这个状态有多好

**Discounting** gives less weight to rewards received further in the future

**Q-value**  $Q_{\pi}(s,a)$  the expected long-term return of taking an action in a state 在这个状态做某个动作有多好

**Stationary preferences** if shifting two reward sequences by the same amount of time does not change their order  
同样的时间变化不会改变两个方案之间的相对偏好

**Additive utility**  $U=r_1+r_2+\dots+r_n$  the sum of all rewards in a sequence

**Discounted utility**  $U=r_1+\gamma r_2+\gamma^2 r_3+\dots$  the weighted sum of rewards, where future rewards receive smaller weights

---

## 2. MDP 求解

**Bellman Equation** Current value = immediate reward + discounted future value

$$U(s) = R(s, a, s') + \gamma U(s')$$

**Value Iteration** repeatedly updates the utility of each state using the Bellman equation until the values converge

$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma U(s')] \quad U(s) = \max_a Q(s, a)$$

**policy extraction** Policy extraction selects the action with the highest expected value in each state.

$$\pi(s) = \arg \max_a Q(s, a)$$

1. Initialize all utilities. 2. Update each state using the Bellman equation. 3. Repeat until the utilities stop changing 4. Extract the best action for each state

**Policy Iteration** alternates between evaluating the current policy and improving it until the policy becomes stable

$$U^{\pi}(s) = \sum_{s'} T(s, \pi(s), s') [R + \gamma U^{\pi}(s')] \quad \pi_{\text{new}}(s) = \arg \max_a U^{\pi}(s)$$

1. Initialize a policy. 2. Evaluate the policy. 3. Improve the policy 4. Repeat until the policy stops changing

---

## 3. Reinforcement Learning 基础

**Reinforce learning** a method in which an agent learns a policy through interaction with the environment and reward feedback

**Model-Based** Learning learns a model of the transition and reward functions and uses it for planning. T R

**Model-Free** Learning learns values or policies from experience without learning an environment model. U Q Policy  
Model-based learns how the world works. Model-free learns what action works.

---

## 4. Passive Reinforcement Learning

**Passive Reinforcement Learning** estimates state values while following a fixed policy.

**Direct Evaluation** Complete the episode and average the returns

Run an episode  $\rightarrow$  calculate returns  $\rightarrow$  store them  $\rightarrow$  take the average

**Temporal Difference Learning** updates a state value after each transition using the immediate reward and the estimated value of the next state.

1. Initialize the state values. 2. Follow the fixed policy and take an action. 3. Observe the reward and next state.  
4. Update the current state value. 5. Move to the next state and repeat.

---

## 5. Active Reinforcement Learning

**Active Reinforcement Learning** requires the agent to explore actions and learn an optimal policy

**Q-Learning** requires the agent to explore actions and learn an optimal policy

1. Initialize all Q-values. 2. Choose an action in the current state. 3. Take the action and observe the reward and next

state. 4. Update the Q-value using the reward and the largest Q-value of the next state. 5. Move to the next state and repeat until the terminal state. 6. Finally, choose the action with the largest Q-value in each state.

**Learning rate** controls how much the parameters change after each experience

**off-policy learning** learns a target policy while following a different behavior policy

**on-policy** learns the same policy that the agent follows

---

## 6. Exploration 与 Exploitation

**exploration** means trying uncertain actions to obtain more information

**exploitation** means choosing the action with the highest estimated value

**$\epsilon$ -greedy** selects the best-known action most of the time and selects a random action with probability  $\epsilon$

1. Generate a random number between 0 and 1. 2. If the number is less than  $\epsilon$ , choose a random action. 3. Otherwise, choose the action with the largest Q-value. 4. Take the action and observe the reward. 5. Update the Q-value of the action. 6. Repeat the process.

**exploration functions** gives preference to actions that have not been tried sufficiently often

$N_e$ : 最少探索次数

$R^+$ : 一个较大的乐观价值

$$f(Q(s, a), N(s, a)) = \begin{cases} R^+, & N(s, a) < N_e \\ Q(s, a), & N(s, a) \geq N_e \end{cases}$$

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[ r + \gamma \max_{a'} f(Q(s', a'), N(s', a')) \right]$$
 通过优先选择尝试次数少的动作进行探索

**Regret** is the difference between the reward obtainable from the optimal action and the reward actually obtained

---

## 7. Approximate Reinforcement Learning

**Approximate Q-Learning** uses a function with a small number of parameters to estimate values or policies in large state spaces.

1. Choose features and initialize the weights. 2. Calculate the approximate Q-value for each action and choose an action. 3. Take the action and observe the reward and the next state. 4. Calculate the error and update the weights. Repeat the process.

**Generalization** allows learning from one state to affect similar states

**Feature-based representation** is a measurable property of a state or a state-action pair

**Policy Search** directly adjusts the parameters of a policy to increase expected reward